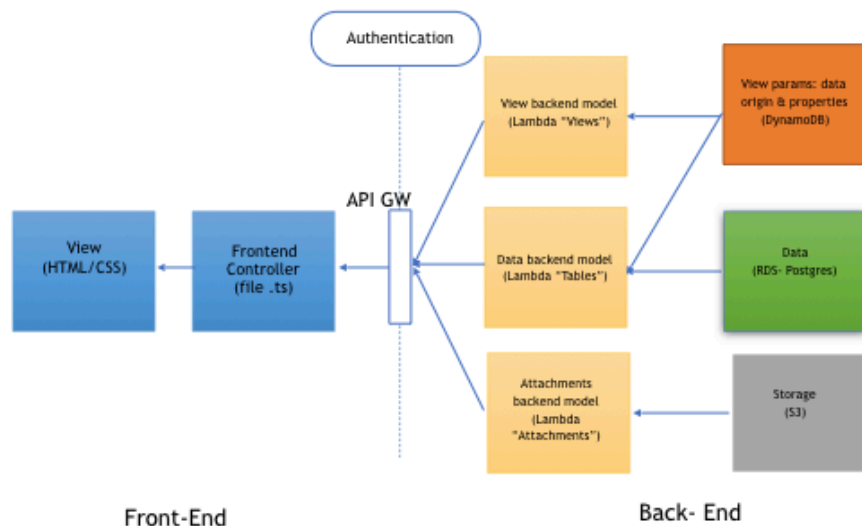


GoRiCo backend data format and frontend views

1. The (new) architecture



In order to visualize the correct view, the front-end has to take in account both the View model (coming from “Views” lambda) and the actual data coming from “Tables” lambda.

View data are in fact equivalent (with much less parameters) to what described as “Proprietà maschera” and “Proprietà singolo elemento” in the Access section below.

The two types of property are properly nested using DynamoDB JSON format as specified in the next paragraphs.

“Tables” lambda is accessing to PostgreSQL database to retrieve actual data, however it also needs to access to DynamoDB to retrieve the full table queries and/or the row queries (“origine riga” in Access terminology) associated with a certain field.

In terms of Views (“maschere”) we will have 3 types of formats:

- 1) “Table” or **TableView** (simplified Access’ “maschere continue”)- a simple table whose columns are specified by the view model and rows are filled with data coming from the data model. The table is a read-only element, a row can be selected but not edited. A table typically also comes with an “advanced search” form, which enables a filtering of the rows depending on some column values/conditions
- 2) “Single row form” or **FormView** (“maschere singole”)- a set of field related to a single record of the table. Properties can be read-only or read/write.
- 3) “Multiple rows form” or **FormTableView** (“maschere continue”) - a combination of the two previous view types, providing an array of forms

Here below a couple of view examples. The first image shows a table view:

Filter

Advanced Search

Items per page: 50 1 - 22 of 22

+ Quick Add

	Codice	Descrizione	Responsabile	Centro Superiore
10	IT	Information Technology	m.azzini - Azzini Mauro	Consiglio di amministrazione
11	HR	Risorse Umane	e.bonazzoli - Bonazzoli Elisa	Consiglio di amministrazione
12	Amministrazione	Amministrazione	d.pini - Pini Dario	Consiglio di amministrazione
16	Crediti	Crediti area legale	g.bertini - Bertini Gianluigi	Consiglio di amministrazione
17	Crediti_finanziari	Crediti finanziari	federico.guarneri - Guarneri Federico	Consiglio di amministrazione
18	Crediti commerciali	Crediti commerciali	e.buizza - Buizza Ettore	Consiglio di amministrazione
19	small_ticket	Small tickets	a.valtolina - Valtolina Ambrogio	Consiglio di amministrazione
20	Transaction&Advisory	Transaction & Advisory	m.zampatti - Zampatti Mara	Consiglio di amministrazione
22	Sicurezza	Coordinamento sicurezza e o.c.	p.bindi - Bindi Paolo	Consiglio di amministrazione
23	Privacy	Responsabile privacy clienti	e.pagani - Pagani Elena	Consiglio di amministrazione

The second image shows a single row form:

“Pulsante di opzione” reported in the Access section is here represented by “**radiobutton**” element, while “Pulsante a scelta multipla” is a set of “**checkboxes**” (see also the DynamoDB JSON format specified below)

ID

2

Codice

CdA

Descrizione

Consiglio di amministrazione

Centro gestionale di livello superiore

Responsabile

francesco.guarneri - Francesco Guarneri

☐ Supervisore di tutti i sondaggi

☐ Gestore di tutti i modelli di test

Centri gestionali Elenco Personale Procedure aziendali

+ Quick Add

Full View / Reduce

	Codice	Descrizione	Responsabile	Centro Superiore
10	IT	Information Technology		
11	HR	Risorse Umane		
12	Amministrazione	Amministrazione		
16	Crediti	Crediti area legale		

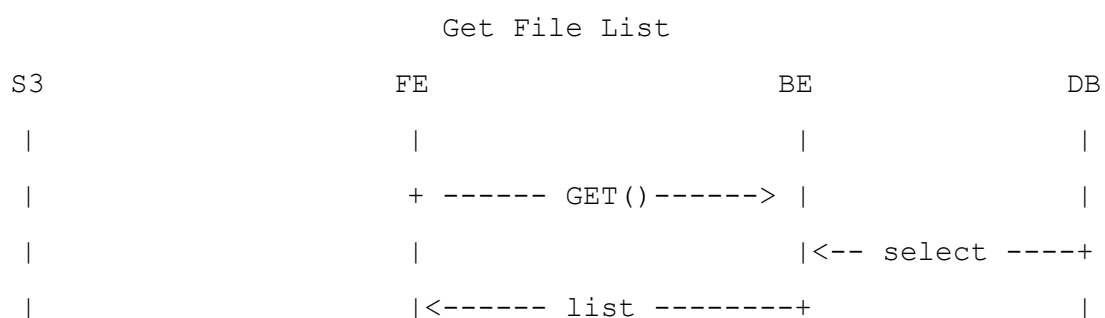
2. Back-end API

Backend REST API is provided by API GW by means of AWS Amplify tool. All APIs calls are authenticated against the token provided when logging in. Every endpoint is associated with a lambda with the following association/syntax

Name / HTTP method	REST API Call	Lambda	View Types	Function
get_userdata HTTP GET	https://<endpoint>/user	User	-	Retrieve the authenticated user info
get_menu HTTP GET	https://<endpoint>/menu?keys=<user_keys>	Navigation	-	Retrieve the left menu based on authenticated user and provided keys
get_view HTTP GET	https://<endpoint>/view?entry_name=<dyndb_name>	Views	All types	Retrieves the view type and related visualization properties
get_data HTTP GET	https://<endpoint>/data?entry_name=<dyndb_name>&keys=<table_keys>&search_keys=<search_keys>&form=<form>&new=<new>&dashboard_index=<dashboard_index>	Tables	All types	Retrieves data associated with the view. new= 1 gets an empty (possibly with ID) record (the operation does NOT create a new record in Postgres DB). form=1 indicates that a record form must be retrieved. If search_keys is defined a search form entry is retrieved. new, form and search_keys parameters are mutually exclusive. combobox options within search form are passed with the full table request previously issued. Dashboard_index decodes the queries (if any) related to a dashboards entry.
update_data HTTP POST	https://<endpoint>/data?entry_name=<dyndb_name>&keys=<table_keys>	Tables	Single&Multiple rows views	Update currently shown record. Creates a new record in Postgres DB if not present (new record). Actual values are passed within the call body.
post_event HTTP POST	https://<endpoint>/data?entry_name=<dyndb_name>&event=<event_data>	Tables	Single&Multiple rows views	Used to retrieve the value of a field that has an input event query associated, whose parameters (if any) are resolved using the passed table_keys. event has type {name: String, field: String}
delete_data HTTP DELETE	https://<endpoint>/data?entry_name=<dyndb_name>&keys=<table_keys>	Tables	Single&Multiple rows views	Delete currently shown record.

Name / HTTP method	REST API Call	Lambda	View Types	Function
get_file_data HTTP GET	https://<endpoint>/attach? entry_name=<dyndb_name>&keys=<table_keys> &filename=<filename>	Attach	Single row view	With no filename provided, returns the list of attachments associated with current record view. If filename only is provided returns a temporary signed URL to read a file from S3 storage .
update_file HTTP POST	https://<endpoint>/attach? entry_name=<dyndb_name>&keys=<table_keys> &filename=<filename>&checksum=<checksum>	Attach	Single row view	If filename is not provided a new filename with a temporary signed URL to upload the file into S3 storage is returned. If filename is provided, updates the file details into the DB. If checksum is also provided then verifies that the new file has been correctly stored into S3, and if checksum is ok creates the related entries in the DB, File details are provided with the HTTP body.
delete_file HTTP DELETE	https://<endpoint>/attach? entry_name=<dyndb_name>&keys=<table_keys> &filename=<filename>	Attach	Single row view	Delete a file into the DB returning a S3 signed URL to physically delete the object.
get_report_list HTTP GET	https://<endpoint>/report? entry_name=<dyndb_name>&keys=<table_keys>	Report	Single&Multiple rows views	Returns the list of reports associated with the business object represented by entry_name.
get_report HTTP POST	https://<endpoint>/report? entry_name=<dyndb_name>&keys=<table_keys> &search_keys=<search_keys>&form=<form>	Report	Single&Multiple rows views	Get a temporary link to the requested generated report. The report name is contained in the body of the request. search_keys is optional and defines potential filtering keys in case of full table view. form=1 indicates that the report is specific to a record.

2.1 Sequence diagrams



Get File Details

S3	FE	BE	DB
	+----- GET(filen)--->		
		<-- select -----+	
	<----- details -----+		

Attach New File

S3	FE	BE	DB
	+----- POST()----->		
	<---- URL, filen -----+		
<--- put(URL)-----+			
	+POST(filen,chsum)->		
		+-- insert ---->	
	<---- OK/NOK -----+		

Update File Details

S3	FE	BE	DB
	+----POST(filen)----->		
		+---- update ---->	
	<---- OK/NOK -----+		

Delete File

S3	FE	BE	DB
	+---- DELETE(filen)-->		

```

|                                     | +--- delete ---->|
|                                     |<----- URL -----+
|<--- delete(URL) -----+         |

```

HTTP Body & Response formats

TODO

3. DynamoDB entries format

The JSON format of a DynamoDb entry (of table “views”) is specified below, a question mark as prefix indicates an optional field.

Keys to replace e.g. within a query string can be either put between \$, e.g. \$codice_part\$ **without quotes** even if they are strings (type is automatically taken in account) or between €, e.g. ‘%€descrizione€%’ if you do not want that data types are taken in account (e.g. quotes are NOT automatically added in case of strings).

In case of pre-defined queries (overriding queries built automatically starting from provided form/table view keys), it is possible to define "local keys" within the query strings, which are result of one or more queries that precede the current query in the predefined query list, i.e. they are not provided by the frontend. Local keys are replaced runtime and must be put between £, e.g. £next_group_id£. Also their format is not specified within the DynamoDB entries, which means that quotes have to be explicitly written whenever needed.

```

{
  "entryKey": String,          // DynamoDB primary key (entry)

  ?"origin": String,           // Postgresql underlying table

  ?"businessObjectName": String, // Internal representation of the business object, e.g. used by attachments
                                // tables

  ?"predefinedQueries": [ // set of possible queries, overriding and/or complementing automatic ones built from
                          // provided columns/fields

    {

      "queryString": String,    // Postgres query string

      "operation": String,      // "selectTable" | "selectForm" | "insert" | "update" | "search" | "delete" |
      "duplicate"

      "type": String            // "main" | "preProcessing" | "postProcessing" , only one main query allowed at
      // most for each operation

    }

  ], // predefinedQueries

  ?navigationTarget: String     // (table view only) target table form to retrieve when a record is selected (if
  // not specified, defaults to self)

  ?"outputEvent": {            // (form/formtable view only) global event triggered once data is (re)loaded

    "eventName": String        // name of the event

  },

  ?"inputEvents": [            // (form/formtable view only) input events associated with the table

    {

```

```

        eventName: String,                // list of event names to listen to
        actionType: String,               // one among "show" | "navigate"
        ?actionTarget: {                 // in case of "navigate" the target info
            "name": String,               // business Object (table name)
            "type": String                // one among "table" | "form" | "tableform"
        }
    ],
    ?"dashboards": [                    // array of dashboard related configuration
        {
            dataSource: {
                data: []                 // set an empty array here
            },
            localization: String,        // 'it' | 'en'
            ?colors: [String],           // array of colors, one for each cell from left to right, from top to bottom
            ?colorsQuery: String,        // alternative to colors, same order as above, however data got from DB
            options, conditions, slice   // see https://www.webdatarocks.com/doc/ , can be generated visually enabling
            the header in the dashboard
        }
    ],
    ?"table_keys": [                    // array of view fields to retrieve/display in Table View
        {
            "isHidden": Boolean,
            "isPrimary": Boolean,        // Primary keys must be always provided with the table
            "key": String,               // postgresql table column name
            ?"queryFunc": String,        // optional postgresql query format for the column, e.g. for
            "key": "responsabile" ->    "entrasp.anagrafiche_id_codcognnome($codice_part$,id_responsabile) as responsabile
            "label": String,             // displayed key name
            "format": {
                "dataType": String,      // "text" | "date" | "number" | "boolean"
                ?"value": String | Number | Boolean // default value
            }
        }
    ], // table_keys
    ?"search_keys": [                  // array of fields of the search form
        {
            "fieldName": String,        // form field name, might or not correspond to a postgres column
            "newLine": Boolean,         // new line with the next field
            ?"size": Number,            // size in 1/10 of available space, only considered if sharing line with other
            keys. If not defined take all space left by the other keys
            "label": String,            // displayed key name
            "queryCond": String,        // postgres query condition (after WHERE clause), mandatory to link w/ a
            Postgres column
            "format": {                 // → DataFormat type
                "viewType": String,      // form view type, one among "input" |
                "combobox" | "checkbox" | "radiobutton"

```

```

        ?"dataType": String,                                // "text" | "date" | "number" | "boolean",
only if viewType="input"

        ?"value": String | Number | Boolean                // default value

        ?"options": [                                     // in case of combobox | radiobutton
            {
                "id": Number,                               // combobox entry ID
                "name": String                              // displayed entry value
            }
        ],

        ?"comboQuery": String,                             // combobox query, returns an array of [{"id":
Number, "name": String}]

    } // format

}

],

?"form_keys": [      // array of view fields to show into single/multiple records form view
{
    "isHidden": Boolean,

    ?"autoGenerate": Boolean, // only if primary key, generated by backend for new entry-> typically table ID

    "readOnly": Boolean,

    "newLine": Boolean,      // new line with the next field

    ?"size": Number,         // size in 1/10 of available space. If defined, must be defined for all keys on
the same line. If not defined, space is split equally

    ?"style": { // set of style attributes
        ?"background_color": String // as per specification @ https://www.w3schools.com/cssref/css_colors.asp -
defaults to "transparent"
        ?"font_color": String      // as above
    },

    "isPrimary": Boolean,      // Primary keys must be always provided with the table

    "key": String,             // postgresql table column name, unless it is composed by sub keys

    ?"subKeys": [              // possible multiple keys concurring to combobox ID field (optional, in this
case the sub keys represent the postgresql table column names)
    {
        "key": String,

        "dataType": String      // "text" | "date" | "number" | "boolean"
    }
    ],

    "label": String,           // displayed key name

    ?"sameOrigin": Boolean      // when false indicates the key is not part of table columns, so it is not
retrieved by automatic query. Default is yes

    ?"queryFunc": String,       // optional postgresql query for the column, exclude the field from automatic
query. Must NOT be used together with sameOrigin.

    ?"evalFunc": String,        // expression to evaluate the value, typically with £ contoured local
parameters. Can or cannot have same origin.

    "format": {                // → DataFormat type

        "viewType": String,     // form view type, one among "input" | "textarea" | "combobox" |
"checkbox" | "radiobutton" | "button"

        ?"dataType": String,    // "text" | "date" | "number" | "boolean" // in case of input only

        ?"value": String | Number | Boolean                // default value
    }
}

```



```

        ?"options": [                                // in case of combobox | radiobutton
            {
                "id": Number,                        // combobox entry ID
                "name": String                       // displayed entry value
            }
        ],

        ?"comboQuery": String,                      // combobox query fields, returns an array of
        [{"id": Number, "name": String}]

        ?"validations": [                          // only for "input" type, check on entered data
            {
                message: String,                    // message to display if condition not met
                name: String,                       // One of "required" | "pattern"
                validator: String                   // e.g. Validators.required, /^[a-zA-Z0-9&_ ]+$

            }
        ] // validations
    }, // format

    ?"outputEvent": {                               // event triggered by the field every time its value changes
        eventName: String                          // event name pushed to other fields, must be unique
    },

    ?"inputEvents": [                               // input events associated with the field
        {
            eventName: String,                      // event to listen to
            actionType: String,                     // "update" | "update_style" | "show" | "query" | "query_style"
            ?styleAttribute: String                 // "background_color" | "font_color" if actionType=update_style
            // or query_style, mandatory if actionType is one of the previous two
            ?updateFunc: String                    // expression to evaluate the value in case of update/
            // update_style event, typically with £ contoured local parameters.
            ?queryFunc: String                     // query to perform in case of query/query_style action. If it
            // is a query_style then the returned key must have name <key>_<styleAttribute>

        }
    ] // inputEvents
}], // form_keys

?"subTables": [                                    // one entry for each tab of the single form view
{
    "label": String,                               // displayed search field name
    "entryKey": String,                            // DynamoDB primary key (entry) of subtable
    ?"type": String,                               // "table" | "tableForm", if not present "table" is assumed
    ?"isHidden": Boolean                           // if not defined defaults to false
    "keys": [
        {
            "parent": String,                      // parent table key, condition on subtable is WHERE son_key = parent_key_value
            "son": String                          // subtable key
        }
    ]
}, // keys

```

```

    ?"inputEvents": [                                // input events associated with the tab
        {
            eventName: String,                        // list of event names to listen to
            actionType: String                        // "show" | "notShow"
        }
    ]
}
] // subTables
}

```

5. Permissions

Within “profiles” DynamoDb table, user profiles are reported. Each profile has a set of permissions to access/visualize/modify the different business objects and/or their fields. Also, some view related permissions are addressed. Finally some operations related permission are addressed.

In the spec below, which addresses an entry in table “profiles”, “deny” permissions have higher priority with respect to “allow” ones. That is, “allow” kind of permissions are considered first, then “deny” ones are taken in account, the latter overriding the first in case of conflict.

Permissions related to the left menu MUST be coherent with permissions related to the respective underlying tables to avoid a manual bypass.

```

{
    menu: {                                           // filter in/out left menu entries
        allow: [String], // set of entries allowed, '*' element -> all entries, empty array -> none
        deny: [String]   // as above but denied
    },
    tables: {
        allow: [String], // set of table entries allowed, '*' element -> all entries, empty array ->
        none
        deny: [String], // as above but denied
        ?readOnly: [String], // set of table entries that are read only (form_view)
        ?table_keys: [{ // set permissions at single key level, both in terms of view and data
            entry: String // table name
            hide: [String] // set of keys to hide, overrides isHidden in table view
        settings
        }],
        ?form_keys: [{
            entry: String, // table name
            hide: [String], // set of keys to hide, overrides isVisible in form view
        settings
        }],
        ?hideQuery: String, // query returning true/false to enable hide filtering, can
        use ?anagrafica_id? if defined for the current user
        readOnly: [String], // set of keys to make read only, overrides readonly
        in form view settings
        ?readOnlyQuery: String // query returning true/false to enable readonly
        filtering, can use ?anagrafica_id? if defined for the current user
    }
}

```

```
        }}  
    }  
    // tables  
}
```

Sommario

1. The (new) architecture	1
2. Back-end API	3
2.1 Sequence diagrams	4
5. Permissions	10
1. PROPRIETA' MASCHERA.....	13
1.1. TIPI MASCHERE	13
1.2. SEZIONE FORMATO	14
1.3. SEZIONE DATI	15
1.4. SEZIONE EVENTO	16
1.5. SEZIONE ALTRO	17
2. PROPRIETA' SINGOLO ELEMENTO BOX.....	18
2.1. TIPI ELEMENTI	18
2.2. SEZIONE FORMATO	20
2.3. SEZIONE DATI	21
2.3.1 BOX NORMALE.....	21
2.3.2 COMBO BOX (CASELLA COMBINATA).....	21
2.4. SEZIONE EVENTO	22
2.1. SEZIONE ALTRO	22

1. PROPRIETA' MASCHERA

1.1. TIPI MASCHERE

In access esistono n tipi di elementi di maschera. I due che ci interessano sono elencati di seguito:

Finestra delle proprietà
Tipo di selezione: Maschera

Maschera

Formato Dati Evento Altro Tutte

Etichetta	
Visualizzazione predefinita	Maschera singola
Consenti visualizzazione Maschera	Maschera singola
Consenti visualizzazione Foglio dati	Maschere continue
Consenti visualizzazione Layout	Foglio dati
Tipo immagine	Dividi maschera
Immagine	(nessuna)
Espansione immagine	No
Allineamento immagine	Allineato al centro
Modalità Ridimens. immagine	Ritaglia

Maschere singole

articoli_normativi

articoli_normativi

codice_articolo_normativo	Art. 173-bis
codice_articolo_normativo_parent	
descrizione	Chiunque, allo scopo di conseguire per sé o per altri un ingiusto profitto, nei prospetti richiesti per l'offerta al pubblico
rubrica	Falso in prospetto
rif_esterno_url	

Maschere continue

rischi_articoli_normativi X

Rischio

▶	603	
	603	
	604	
	604	
	605	
	605	
	0001	dfsfsdfds
	606	
	0002	Guasti alle apparecchiature hardware

1.2. SEZIONE FORMATO

Formato	Dati	Evento	Altro	Tutte
Etichetta				
Visualizzazione predefinita				Maschera singola
Consenti visualizzazione Maschera				Sì
Consenti visualizzazione Foglio dati				No
Consenti visualizzazione Layout				Sì
Tipo immagine				Incorporata
Immagine				(nessuna)
Espansione immagine				No
Allineamento immagine				Allineato al centro
Modalità Ridimens. immagine				Ritaglia
Larghezza				28,069cm
Allineamento al centro automatico				No
Dimensioni automatiche				Sì
Adatta allo schermo				Sì
Stile bordo				Dimensionabile
Selettori record				Sì
Pulsanti spostamento				Sì
Didascalia spostamento				
Linee di divisione				No
Barre scorrimento				Entrambe
Casella menu controllo				Sì
Pulsante chiusura				Sì
Pulsanti ingrand./riduzione				Entrambi
Spostabile				No
Dimensione maschera divisa				Automatica
Orientamento maschera divisa				Foglio dati in alto
Barra di divisione maschera divisa				Sì

Orientamento maschera divisa	Foglio dati in alto
Barra di divisione maschera divisa	Sì
Foglio dati maschera divisa	Consenti modifiche
Stampa maschera divisa	Solo maschera
Salva posizione barra di divisione	Sì
Foglio dati sec. espanso	No
Altezza foglio dati sec.	0cm
X griglia	10
Y griglia	10
Caratteri stampante	No
Orientamento	Da sinistra a destra
Origine tavolozza	(Predefinita)

1.3. SEZIONE DATI

Finestra delle proprietà

Tipo di selezione: Maschera

Maschera 

Formato Dati Evento Altro Tutte

Origine record	articoli_normativi
Tipo Recordset	Dynaset
Recupera impostazioni predefinite	Sì
Filtro	
Filtra al caricamento	No
Ordina per	
Ordina al caricamento	Sì
Attendi post-elaborazione	No
Immissione dati	No
Consenti aggiunte	Sì
Consenti eliminazioni	Sì
Consenti modifiche	Sì
Consenti filtri	Sì
Blocco record	Nessun blocco

1.4. SEZIONE EVENTO

Maschera ▼

Formato
Dati
Evento
Altro
Tutte

Su corrente	
Su caricamento	
Su clic	
Dopo aggiornamento	
Prima di aggiornare	
Prima di inserire	
Dopo inserimento	
Prima di conferma eliminaz.	
Su eliminazione	
Dopo conferma eliminazione	
In attesa	
Su attivato	
Su disattivato	
Su doppio clic	
Su pulsante mouse giù	
Su pulsante mouse su	
Su mouse spostato	
Su tasto su	
Su tasto giù	
Su pressione	
Su annullamento	
Su apertura	
Su chiusura	
Su ridimensionamento	
Su attivazione	
Su disattivazione	
Su scaricamento	

Su scaricamento	
Su errore	
Su rotellina mouse	
Su filtro	
Su applicazione filtro	
Su timer	
Intervallo timer	0
Su modifica selezione	
Prima di rendering	
Dopo rendering finale	
Dopo rendering	
Dopo layout	
Su connessione	
Su disconnessione	
Prima di query	
Su query	
Su modifica dati	
Su modifica set di dati	
Dopo esecuzione comando	
Su comando prima di esecuzione	
Su comando abilitato	
Su comando selezionato	
Su modifica visualizzazione	
Su modifica tabella pivot	
Prima di suggerimento su schermo	
Anteprima tasti	No

1.5. SEZIONE ALTRO

Finestra delle proprietà

Tipo di selezione: Maschera

Maschera 

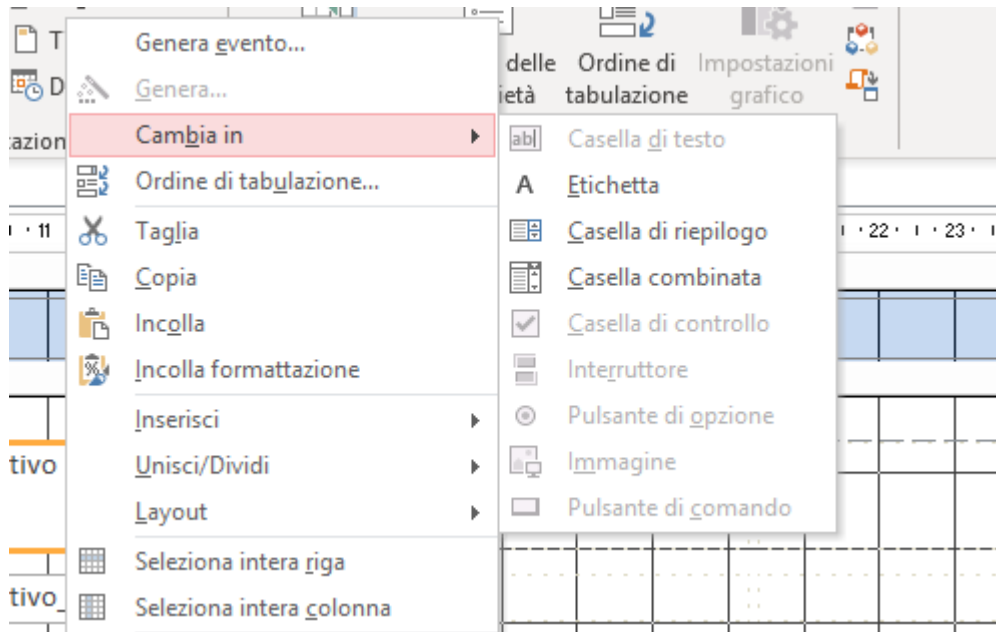
Formato Dati Evento Altro Tutte

Popup	No
A scelta obbligatoria	No
Sequenza	Tutti i record
Nome barra multifunzione	
Barra degli strumenti	
Menu scelta rapida	Sì
Barra menu	
Barra menu scelta rapida	
File Guida	
ID argomento Guida	0
Possiede modulo	No
Usa dimensione foglio predefinita	No
Stampa laser rapida	Sì
Tag	

2. PROPRIETA' SINGOLO ELEMENTO BOX

2.1. TIPI ELEMENTI

in access esistono n tipi di elementi di seguito elencati.



Quelli che a noi interessano sono:

- Casella di testo →

codice_articolo_normativo

Art. 173-bis

- Casella combinata (combo box) →

articoli_normativi

codice_articolo_normativo	D. Lgs. 24 febbraio 1998, n. 58
codice_articolo_normativo_parent	Delibera CONSOB n. 11522
descrizione	D.Lgs. n. 231/01
rubrica	D.lgs 30 giugno 2003, n. 196
rif_esterno_url	Delibera CONSOB n. 11971
	Delibera CONSOB n. 14015
	Decreto 24 maggio 1999, n. 228
	D.Lgs 30 giugno 2003, n. 196
	Legge n. 197 5 luglio 1991
	Delibera CONSOB n. 14015
	Delibera CONSOB n. 16190
	Codice civile
	Codice penale
	Decreto legislativo 25 luglio 1998, n. 286
	Legge 6 febbraio 1980, n.15
id_testo_normativo	Legge 10 maggio 1976, n. 342
	D. Lgs. 24 febbraio 1998, n. 58
	Testo Unico dell'intermediazione finanziar

- Pulsante di opzione;

Tipo allegato previsto

.1) La Società distribuisce contratti assicurativi?

☐ SI
☐ NO
☒ Parzialmente

- Pulsante a scelta multipla;

Crea segnalazione Associa segnalazione

.2) Indicare il nome della cassetta sulla quale è stato effettuato il back-up.

☐ Si
☒ Parzialmente
☐ No
☒ Non applicabile

Crea segnalazione Associa segnalazione

Di seguito le proprietà dei singoli elementi

2.2. SEZIONE FORMATO

Formato
Dati
Evento
Altro
Tutte

Formato	
Posizioni decimali	Automatiche
Visibile	Sì
Visualizza selezione data	Per date
Larghezza	7,619cm
Altezza	1,429cm
Da margine superiore	0,608cm
Sinistra	6,64cm
Stile sfondo	Normale
Colore sfondo	Sfondo 1
Stile bordo	Continuo
Spessore bordo	Sottilissimo
Colore bordo	Sfondo 1, Più scuro 35%
Aspetto	Piatto
Barre scorrimento	Nessuna
Tipo carattere	Calibri (Corpo)
Dimensione carattere	11
Allineamento testo	A sinistra
Spessore carattere	Normale
Carattere sottolineato	No
Carattere corsivo	No
Colore primo piano	Testo 1, Più chiaro 25%
Interlinea	0cm
Colleg. ipertestuale	No
Visualizza come collegamento ipertestuale	Nei collegamenti ipertestuali
Destinazione collegamento ipertestuale	
Stile griglia in alto	Trasparente

Stile griglia in alto	Trasparente
Stile griglia in basso	Trasparente
Stile griglia a sinistra	Trasparente
Stile griglia a destra	Trasparente
Colore griglia	Sfondo 1, Più scuro 35%
Larghezza griglia in alto	1 pt
Larghezza griglia in basso	1 pt
Larghezza griglia a sinistra	1 pt
Larghezza griglia a destra	1 pt
Margine superiore	0,039cm
Margine inferiore	0,039cm
Margine sinistro	0,078cm
Margine destro	0,078cm
Spaziatura interna in alto	0,053cm
Spaziatura interna in basso	0,265cm
Spaziatura interna a sinistra	0,053cm
Spaziatura interna a destra	0,053cm
Ancoraggio orizzontale	A sinistra
Ancoraggio verticale	In alto
Espandibile	No
Riducibile	No
Visualizza	Sempre
Ordine di lettura	Contesto
Allineam. barra di scorrimento	Sistema
Forme dei numerali	Sistema
Lingua tastiera	Sistema

2.3. SEZIONE DATI

2.3.1 BOX NORMALE

codice_articolo_normativo

▼

Formato

Dati

Evento

Altro

Tutte

Origine controllo	codice_articolo_normativo
Formato testo	Testo normale
Maschera di input	
Valore predefinito	
Valido se	
Messaggio errore	
Ricerca con filtro	Impostazione predefinita database
Abilitato	Sì
Bloccato	No

2.3.2 COMBO BOX (CASELLA COMBINATA)

id_testo_normativo

▼

Formato

Dati

Evento

Altro

Tutte

Origine controllo	id_testo_normativo
Origine riga	SELECT testi_normativi.id_testo_normativo, testi_normativi.riferimenti, testi_normativi.descrizione FROM testi_normativi;
Tipo origine riga	Tabella/query
Colonna associata	1
Solo in elenco	Sì
Consenti modifiche a elenco valori	Sì
Maschera di modifica voci di elenco	
Eredita elenco valori	Sì
Mostra solo valori origine riga	No
Maschera di input	
Valore predefinito	
Valido se	
Messaggio errore	
Abilitato	Sì
Bloccato	No
Espansione automatica	Sì

2.4. SEZIONE EVENTO

Formato

Dati

Evento

Altro

Tutte

Su clic	
Prima di aggiornare	
Dopo aggiornamento	
In attesa	
Su modifica	
Su attivato	
Su disattivato	
Su doppio clic	
Su pulsante mouse giù	
Su pulsante mouse su	
Su mouse spostato	
Su tasto giù	
Su tasto su	
Su pressione	
Su INVIO	
Su uscita	
Su annullamento	

2.1. SEZIONE ALTRO

Finestra delle proprietà

Tipo di selezione: Casella di testo

Formato

Dati

Evento

Altro

Tutte

Nome elemento	codice_articolo_normativo
Nome etichetta	Etichetta0
Didascalia foglio dati	
Funzione tasto INVIO	Predefinita
Testo descrizione controllo	
Ordine spostamento	0
Seleziona con tabulazione	Sì
Testo barra stato	
Barra menu scelta rapida	
ID argomento Guida	0
Tabulazione automatica	No
Verticale	No
Consenti correz. automatica	Sì
Mantieni modalità IME	No
Modalità IME	Nessun controllo
Modalità frase IME	Nessuna conversione
Tag	